

Executive summary: YAML IR vs HTML for slide generation research

Context

This repository is built around a declarative YAML/JSON slide IR. `slidekit` parses YAML and JSON deck files, validates them as a `DeckIR`, resolves deterministic geometry, lints the result, and emits `.pptx` or `.pdf`. The repo currently contains 40 `.yaml` files and 6 `.yml` files, including 39 YAML example decks under `examples/`.

The research bibliography in `docs/REFERENCES.md` is mostly about computational aesthetics and automated layout evaluation. Several papers evaluate slides or layouts via rendered HTML/screenshots/pixels, while `slidekit` intentionally computes comparable checks from source geometry and structured theme data.

Where this repo uses YAML

- Deck authoring examples use YAML as the primary human/agent-facing format. A typical deck declares `version`, `theme`, `page_numbers`, and `slides`, then chooses component keys such as `title-slide`.
- `board.yaml` is the source of truth for the active kanban board, rendered into `BOARD.md` by `scripts/render_board.py`.
- The parser accepts `.yaml`, `.yml`, and `.json`; YAML failures are reported as parse errors, and validation failures are converted into field-specific repair messages for agents.
- The schema enforces typed components, safe fonts, type-scale bounds, and theme roles before layout or rendering.

YAML/source-first approach: pros

1. **Deterministic and auditable.** YAML maps directly to validated IR objects and deterministic layout geometry, so failures can be traced to fields and components rather than screenshots.
2. **Agent-friendly edit surface.** Agents can patch small semantic fields (`title`, `items`, `theme.palette.primary`) instead of manipulating nested DOM/CSS or visual pixels.
3. **Strong validation.** Pydantic catches unsafe fonts, invalid colors, missing fields, and component spelling before any deck is emitted.
4. **Low-token, low-render workflow.** The README positions this as a compile-and-lint loop rather than a render-and-inspect loop.
5. **Format-neutral output.** One source deck can emit `.pptx`, native `.pdf`, resolved JSON, and schema artifacts.
6. **Better multi-agent handoff.** Brand, story, and deck-builder agents can exchange structured blocks instead of opaque rendered output.

YAML/source-first approach: cons

1. **Less expressive than HTML/CSS.** YAML IR only supports the components and slots that `slidekit` implements.
2. **Approximate for pixel-only aesthetics.** Edge density, photographic salience, brightness contrast, and rendered texture require raster or browser measurements.
3. **Custom layout engine burden.** The repo must maintain its own component layouts, font metrics, linter, emitters, and golden tests.
4. **Research comparability gap.** Many benchmarks evaluate rendered images, so `slidekit` needs careful mapping from pixel metrics to geometry/theme analogues.
5. **Learning curve.** Users must learn the `slidekit` component vocabulary rather than generic web layout conventions.

HTML/render-first research approach: pros

1. **High visual expressiveness.** HTML/CSS can represent arbitrary layouts, typography, effects, and responsive behavior.
2. **Close to what evaluators see.** Screenshot/pixel metrics measure final rendered appearance, including browser font fallback and raster details.
3. **Broad tooling.** Browsers, DOM inspection, Playwright, CSS layout, and screenshot-based ML pipelines are mature.
4. **Pixel-only metrics.** Research methods can compute edge density, colorfulness, brightness contrast, image salience, and visual complexity from screenshots.
5. **Easy benchmark interop.** Many slide-generation and UI-aesthetics papers already operate on rendered artifacts.

HTML/render-first research approach: cons

1. **Render dependency.** Results depend on browser version, fonts, viewport, device scale, and screenshot pipeline.
2. **Harder semantic edits.** Once content is mixed with DOM/CSS, changing “the third agenda item” can be less reliable than editing structured YAML.
3. **More failure modes.** Visual overlap or overflow may appear only after rendering, requiring screenshot inspection or DOM measurement.
4. **Less constrained generation.** HTML lets agents create arbitrary structures, which is powerful but can reduce consistency and lintability.
5. **Heavier workflows.** Render-and-inspect loops consume more compute and, for multimodal agents, more tokens.

Recommendation

Use YAML/source-first for production slide generation in this repo. It best matches slidekit's goals: deterministic geometry, schema validation, lint-clean builds, and agent-readable fixes.

Use HTML/render-first methods selectively for research calibration and for metrics that genuinely require pixels: photographic salience, edge density, final raster colorfulness, and visual complexity. The best long-term approach is hybrid: keep YAML as the source of truth, emit deterministic geometry for linting, and reserve rendered HTML/PDF screenshots for CI drift checks or human-correlation studies.